

UNIFACENS
Sistemas Operacionais

Bruno Bertolini Maluf
Giovanni Delgado Torres
Guilherme Ferrari Assad Crudo
Guilherme Serafim
Matheus Belloto

SENTINELA.SH – MONITOR INTELIGENTE DE RECURSOS DO SISTEMA

Sorocaba/SP
2026

SUMÁRIO

1 INTRODUÇÃO	3
2 OBJETIVO DO EXPERIMENTO	3
3 MATERIAIS UTILIZADOS	4
4 PROCEDIMENTO EXPERIMENTAL	
5	
5 RESULTADOS	
9	
6 CONCLUSÃO	
10	
REFERÊNCIAS	
11	

1 INTRODUÇÃO

O presente trabalho tem como propósito o desenvolvimento de um script em Bash denominado sentinela.sh, com o objetivo de atuar como um monitor inteligente de recursos e de integridade de diretórios críticos em sistemas Linux. O monitoramento de recursos computacionais é uma prática essencial na administração de sistemas operacionais, pois permite identificar gargalos de desempenho, prevenir falhas e garantir a segurança do ambiente.

A solução proposta faz uso de comandos nativos do ambiente Unix/Linux, como ps, free, find, renice, mv e chmod, integrando-os de forma automatizada e contínua. O script é capaz de verificar permissões de execução, monitorar processos com alto consumo de CPU, gerenciar arquivos suspeitos no diretório /tmp e exibir relatórios periódicos de saúde do sistema, além de responder de maneira adequada a sinais de interrupção do sistema operacional.

2 OBJETIVO DO EXPERIMENTO

O objetivo principal deste experimento é desenvolver um script em Bash funcional e seguro, denominado sentinela.sh, capaz de monitorar continuamente o ambiente do sistema operacional Linux. De forma específica, o script deve:

- a) verificar se está sendo executado com privilégios de superusuário (root);
- b) criar automaticamente um diretório de quarentena para arquivos suspeitos;
- c) identificar e reduzir a prioridade de processos que excedam 80% de uso de CPU;
- d) mover arquivos com mais de 5 MB do diretório /tmp para a quarentena;
- e) exibir relatórios de saúde do sistema a cada 30 segundos;
- f) realizar uma finalização segura ao receber o sinal SIGINT (CTRL+C).

3 MATERIAIS UTILIZADOS

Para o desenvolvimento do script sentinela.sh foram utilizados os seguintes recursos de software e comandos nativos do sistema operacional Linux:

Sistema Operacional: Linux (Ubuntu 22.04 LTS ou compatível), com suporte ao interpretador Bash versão 5.x.

Interpretador Bash: Responsável pela execução do script, utilizando estruturas de controle como if, while e trap, além de comandos de pipeline.

Comando ps: Utilizado para listar os processos em execução e identificar o de maior consumo de CPU, com as opções -eo pid,pcpu,comm --sort=-pcpu.

Comando renice: Empregado para alterar a prioridade de agendamento de um processo, reduzindo-a ao nível máximo de gentileza (+19).

Comando find: Usado para localizar arquivos no diretório /tmp com tamanho superior a 5 MB, com as opções -maxdepth 1 -type f -size +5M.

Comandos mv e chmod: O mv realiza a movimentação dos arquivos para a quarentena; o chmod -x remove as permissões de execução.

Comando free: Fornece informações sobre o uso de memória RAM em megabytes, por meio da opção -m, com filtragem via awk.

Comando id: Verifica o UID do usuário corrente, permitindo ao script confirmar a execução como root (UID 0).

Comando trap: Captura sinais do sistema operacional (SIGINT e SIGTERM), viabilizando uma rotina de encerramento seguro.

4 PROCEDIMENTO EXPERIMENTAL

O desenvolvimento do script sentinela.sh foi estruturado em cinco etapas funcionais, descritas a seguir.

4.1 Verificação de Ambiente e Segurança

A primeira etapa verifica se o script está sendo executado com privilégios de root por meio do comando id -u, que retorna o UID do usuário atual. Caso o valor retornado seja diferente de zero, o script exibe uma mensagem de erro explicativa e encerra a execução com código de saída 1. Em seguida, é verificada a existência do diretório de quarentena /home/\$USER/quarentena. Caso não exista, o diretório é criado automaticamente com o comando mkdir -p.

4.2 Monitoramento de Processos Vilões

A função monitorar_processos() utiliza o comando ps -eo pid,pcpu,comm --sort=-pcpu para listar os processos ordenados por consumo de CPU em ordem decrescente. O processo no topo da lista é extraído via awk. Se o percentual de CPU

for superior a 80%, o comando `renice +19 -p $PID` reduz sua prioridade ao máximo. A ação, incluindo PID, nome do processo, percentual e timestamp, é registrada no arquivo `log_sentinela.txt`.

4.3 Gestão de Arquivos e Integridade

A função `monitorar_tmp()` utiliza `find /tmp -maxdepth 1 -type f -size +5M` para identificar arquivos regulares no diretório `/tmp` que ultrapassem 5 MB. Para cada arquivo encontrado, o script o move para o diretório de quarentena com `mv` e, na sequência, remove sua permissão de execução com `chmod -x`. Cada movimentação é registrada no arquivo de log.

4.4 Relatório de Saúde em Loop

O loop principal executa indefinidamente com intervalo de 30 segundos (`sleep 30`) entre cada iteração. A cada ciclo, as funções `monitorar_processos()` e `monitorar_tmp()` são invocadas. Em seguida, o script exibe no terminal um resumo com o uso atual de memória RAM (extraído de `free -m` via `awk`), o total de processos em execução (`ps aux | wc -l`) e o status do sentinela.

4.5 Finalização Limpa

O comando `trap` captura os sinais `SIGINT` (CTRL+C) e `SIGTERM`. Ao receber qualquer um desses sinais, a função `finalizar()` é chamada, exibindo a mensagem “Sentinela encerrado com segurança”, removendo arquivos temporários criados pelo próprio script (registrados no array `TEMP_FILES`) e encerrando a execução com `exit 0`.

4.6 Código-Fonte do Script `sentinela.sh`

A seguir é apresentado o código-fonte completo do script desenvolvido:

```
#!/bin/bash

# =====
# sentinela.sh - Monitor Inteligente de Recursos do Sistema
# Sistemas Operacionais - 2026
# =====

QUARENTENA="/home/$USER/quarentena"
LOG_FILE="./log_sentinela.txt"
```

```

TEMP_FILES=()

# -----
# 1. VERIFICACAO DE AMBIENTE E SEGURANCA
# -----

if [ "$(id -u)" -ne 0 ]; then
    echo "[ERRO] Este script precisa ser executado como root."
    echo "      Use: sudo ./sentinela.sh"
    exit 1
fi

if [ ! -d "$QUARENTENA" ]; then
    mkdir -p "$QUARENTENA"
    echo "[INFO] Diretorio de quarentena criado: $QUARENTENA"
fi

# -----
# 5. FINALIZACAO LIMPA (trap CTRL+C)
# -----

finalizar() {
    echo ""
    echo "=====
    echo " Sentinela encerrado com seguranca."
    echo "=====
    for f in "${TEMP_FILES[@]}"; do
        [ -f "$f" ] && rm -f "$f"
    done
    exit 0
}

trap finalizar SIGINT SIGTERM

# -----
# 2. MONITORAMENTO DE PROCESSOS VILOES
# -----

monitorar_processos() {
    PROC_INFO=$(ps -eo pid,pcpu,comm --sort=-pcpu | awk 'NR==2')
    PID=$(echo "$PROC_INFO" | awk '{print $1}')
    CPU=$(echo "$PROC_INFO" | awk '{print $2}' | cut -d. -f1)
    NOME=$(echo "$PROC_INFO" | awk '{print $3}')

    if [ "$CPU" -gt 80 ]; then
        renice +19 -p "$PID" > /dev/null 2>&1
        DATA=$(date '+%Y-%m-%d %H:%M:%S')
        MSG="[${DATA}] ALERTA: Processo '$NOME' (PID $PID) com ${CPU}%"
        CPU. nice +19."
        echo "$MSG" >> "$LOG_FILE"
        echo "$MSG"
    fi
}

# -----
# 3. GESTAO DE ARQUIVOS E INTEGRIDADE
# -----

monitorar_tmp() {
    find /tmp -maxdepth 1 -type f -size +5M | while read -r ARQUIVO;
do
    NOME_ARQ=$(basename "$ARQUIVO")
    mv "$ARQUIVO" "$QUARENTENA/$NOME_ARQ"
    chmod -x "$QUARENTENA/$NOME_ARQ"
    DATA=$(date '+%Y-%m-%d %H:%M:%S')

```

```

        MSG="[$DATA] MOVIDO: $ARQUIVO -> $QUARENTENA/$NOME_ARQ"
        echo "$MSG" >> "$LOG_FILE"
        echo "$MSG"
    done
}

# -----
# 4. RELATORIO DE SAUDE (LOOP A CADA 30 SEGUNDOS)
# -----

echo "=====
echo "  SENTINELA ATIVO - Iniciando monitoramento"
echo "=====

while true; do
    monitorar_processos
    monitorar_tmp

    MEM_USADA=$(free -m | awk '/^Mem:/{print $3}')
    TOTAL_PROCS=$(ps aux --no-headers | wc -l)

    echo "-----"
    echo "  Uso de Memoria Atual   : ${MEM_USADA} MB"
    echo "  Total de Processos     : ${TOTAL_PROCS}"
    echo "  Status do Sentinela    : Ativo"
    echo "  Proxima verificacao    : 30 segundos"
    echo "-----"

    sleep 30
done

```

5 RESULTADOS

Durante os testes de execução do script `sentinela.sh`, foram observados os seguintes comportamentos:

a) Verificação de Root: ao ser executado sem privilégios de superusuário, o script exibiu corretamente a mensagem de erro e encerrou a execução, impedindo operações privilegiadas sem a devida autorização.

b) Criação do diretório de quarentena: o diretório `/home/$USER/quarentena` foi criado com sucesso na primeira execução, e nas subsequentes o script reconheceu sua existência sem erros.

c) Monitoramento de processos: em cenário simulado com o comando `stress`, o sentinela identificou corretamente o processo e aplicou `renice +19`, registrando a ação no `log_sentinela.txt` com data e hora.

d) Gestão de arquivos em `/tmp`: arquivos de teste com mais de 5 MB foram detectados, movidos para a quarentena e tiveram a permissão de execução removida, conforme esperado.

e) Relatório de saúde: a cada 30 segundos, o terminal exibiu o resumo com uso de memória, total de processos e status do sentinela, de forma contínua e estável.

f) Finalização com CTRL+C: o script capturou o sinal SIGINT, exibiu a mensagem de despedida e encerrou a execução de maneira limpa, sem processos órfãos ou arquivos temporários.

6 CONCLUSÃO

O desenvolvimento do script sentinela.sh permitiu a aplicação prática de conceitos fundamentais da disciplina de Sistemas Operacionais, como gerenciamento de processos, controle de permissões, sinais do sistema operacional e automação de tarefas administrativas em ambientes Linux.

O script demonstrou ser uma solução eficaz para o monitoramento automatizado de recursos computacionais, sendo capaz de identificar e remediar situações de sobrecarga de CPU, isolar arquivos potencialmente perigosos no diretório temporário e fornecer informações periódicas sobre o estado do sistema de forma integrada e transparente ao administrador.

O uso do comando trap para captura de sinais revelou-se essencial para garantir a robustez de scripts em modo contínuo, evitando interrupções abruptas que comprometam a integridade dos dados ou do sistema.

Como trabalhos futuros, sugere-se expandir o sentinela para monitorar também o uso de disco, a atividade de rede e a integridade de arquivos de sistema via hashes criptográficos, além de integrá-lo a sistemas de notificação como e-mail ou mensagens instantâneas.

REFERÊNCIAS

BARRETT, Daniel J.; SILVERMAN, Richard E.; BYRNES, Robert G. *SSH, The Secure Shell: The Definitive Guide*. 2. ed. Sebastopol: O'Reilly Media, 2005.

KERRISK, Michael. *The Linux Programming Interface*. São Francisco: No Starch Press, 2010.

NEWHAM, Cameron; ROSENBLATT, Bill. *Learning the bash Shell*. 3. ed. Sebastopol: O'Reilly Media, 2005.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. *Fundamentos de Sistemas Operacionais*. 9. ed. Rio de Janeiro: LTC, 2015.

TANENBAUM, Andrew S.; BOS, Herbert. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016.